Ascent
FORMATION

FORMATION PROFESSIONNELLE · COURS APPRENANT

# Cours apprenant

*ReST API — bonnes pratiques et sécurité*

MODULE 5

## Déploiement et supervision des API

CI/CD, observabilité, Prometheus, Grafana, ELK, OpenTelemetry, SLO

CRÉATEUR DU COURS

**Boris Rose**

*Formateur & concepteur du contenu*

# Table des matières

**Avant-propos** **3**

**1 Pipeline CI/CD pour API** **3**
1.1 Cinq étapes essentielles . . . 3
1.2 Stratégies de déploiement . . . 3
1.2.1 Recreate . . . 4
1.2.2 Rolling . . . 4
1.2.3 Blue/Green . . . 4
1.2.4 Canary . . . 5
1.2.5 A/B Testing . . . 6
1.2.6 Shadow (mirror traffic) . . . 6

**2 Observabilité : les 3 piliers** **7**

**3 Logs structurés** **7**
3.1 Pourquoi structurer ? . . . 7
3.2 Bonnes pratiques . . . 7
3.2.1 Format NDJSON . . . 7
3.2.2 Champs standards (recommandation OpenTelemetry) . . . 8
3.2.3 Niveaux de log cohérents . . . 8
3.2.4 Jamais de PII dans les logs . . . 9
3.2.5 Propagation du trace ID . . . 9

**4 Métriques avec Prometheus** **10**
4.1 Architecture . . . 10
4.2 Types de métriques Prometheus . . . 10
4.3 Méthode RED . . . 10
4.4 PromQL — requêtes essentielles . . . 10

**5 Grafana et dashboards** **11**
5.1 Composer un dashboard utile . . . 11

**6 SLO et error budget** **11**
6.1 Alerting basé sur SLO . . . 11

**7 Tracing distribué** **12**
7.1 OpenTelemetry . . . 12

**8 ELK Stack et alternatives** **12**
8.1 Composants ELK . . . 12
8.2 Alternatives . . . 12

**9 Les outils du module** **12**
9.1 Prometheus . . . 12
9.2 Grafana . . . 13
9.3 Elastic Stack / OpenSearch . . . 13
9.4 Grafana Loki . . . 13
9.5 OpenTelemetry . . . 13
9.6 Jaeger, Tempo, Zipkin . . . 13
9.7 ArgoCD, Flux . . . 13

9.8 Trivy, Grype . . . . . 14

**10 Synthèse** **14**

**11 Lexique du module** **14**

**12 Pour aller plus loin** **15**

## Avant-propos

Ce cours est le compagnon écrit du **Module 5**. Il couvre les pipelines CI/CD, les trois piliers de l'observabilité, les SLO, et les outils que vous utiliserez en TP. *Cours conçu et rédigé par Boris Rose, formateur.*

# 1 Pipeline CI/CD pour API

**Définition**

**CI/CD** (*Continuous Integration / Continuous Delivery / Continuous Deployment*) : pratique consistant à automatiser entièrement la chaîne qui va du commit Git jusqu'à la mise en production, en passant par les tests, les scans de sécurité et les déploiements.

## 1.1 Cinq étapes essentielles

1. **Build** : compilation du code, fabrication du *package* déployable (typiquement une image Docker), poussé dans un registre. Doit être *reproductible* : un même commit produit toujours la même image, bit pour bit.
2. **Test** : tests unitaires (fonction par fonction), d'intégration (plusieurs composants ensemble), de contrat (*Pact* : vérifie que producteur et consommateur d'API se comprennent toujours), end-to-end légers (un parcours complet à travers l'application).
3. **Scan** : passage de plusieurs analyseurs de sécurité :
   - **SAST** (*Static Application Security Testing*) : analyse du code source sans l'exécuter. Outils : Semgrep, SonarQube, CodeQL.
   - **SCA** (*Software Composition Analysis*) : scan des dépendances tierces pour repérer les versions avec CVE connues. Outils : Snyk, Dependabot, Trivy.
   - **Scan de secrets** : détection de clés API, tokens, mots de passe oubliés dans le code. Outils : gitleaks, TruffleHog.
   - **DAST** (*Dynamic Application Security Testing*) : test *en cours d'exécution*, en envoyant des payloads d'attaque. Outil : OWASP ZAP.
   - **SBOM** (*Software Bill of Materials*) : inventaire complet de tous les composants logiciels embarqués (équivalent d'une « liste d'ingrédients »). Format SPDX ou CycloneDX. Imposé par l'*Executive Order 14028* US pour les fournisseurs du gouvernement.
4. **Deploy** : déploiement en environnement *staging* (préproduction, données proches du réel), exécution des tests *E2E* (*end-to-end*), puis bascule en production.
5. **Verify** : *smoke tests* (vérifications rapides « l'API répond, le login marche »), vérification que les métriques restent dans les seuils, rollback automatique si les SLO sont cassés.

## 1.2 Stratégies de déploiement

**Définition**

**Déploiement progressif** : technique pour mettre en production une nouvelle version sans interruption de service et avec possibilité de retour arrière rapide. Six grandes stratégies, à choisir selon le compromis entre risque, coût en infrastructure et complexité opérationnelle.

### 1.2.1 Recreate

**Définition**

**Recreate** : on arrête *toutes* les instances de l'ancienne version, puis on démarre la nouvelle. La plus simple, la plus risquée : downtime garanti (quelques secondes à quelques minutes), pas de rollback rapide.

**Exemple**

**Cas typique** : un outil interne utilisé en heures de bureau. Déploiement programmé chaque jeudi 22h. Si le nouveau build est cassé, on doit redéployer manuellement l'image précédente.

**Configuration Kubernetes :**

```
spec:
  strategy:
    type: Recreate
```

### 1.2.2 Rolling

**Définition**

**Rolling update** : remplacement progressif des instances une par une (ou par petits lots). Pas de downtime, peu de surcoût en infra. Stratégie par défaut de Kubernetes. Inconvénient : pendant la transition, les deux versions cohabitent en production simultanément, ce qui peut poser problème si elles sont incompatibles (changement de schéma BDD, etc.).

**Exemple**

**Cas** : API `orders-api` avec 10 pods. La nouvelle version est déployée par lots de 2 pods (20 %) :

```
spec:
  replicas: 10
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 2 # max 12 pods (10 + 2) simultanément
      maxUnavailable: 1 # au pire 9 pods servent pendant la transition
```

**Déroulement** : K8s démarre 2 nouveaux pods → attend leur *readiness* → coupe 2 anciens → 2 nouveaux → . . . jusqu'à 10 pods de la nouvelle version. Durée typique : 3–5 minutes.

### 1.2.3 Blue/Green

**Définition**

**Blue/Green** : deux stacks complètes coexistent. La stack *Blue* sert tout le trafic ; la stack *Green* héberge la nouvelle version mais ne reçoit rien. Une fois Green validée (tests E2E sur la même infra que prod), on bascule *tout* le trafic d'un coup (DNS, load balancer, ingress). Rollback instantané : on rebascule sur Blue. Coût : il faut payer la double infra pendant la transition.

**Exemple**

**Cas** : application e-commerce critique pendant le Black Friday.

1. Stack Blue (v1.4.2) sert 100 % du trafic via `api.shop.com → blue.prod.shop.com`.
2. Stack Green (v1.5.0) déployée en parallèle. On la teste via `green.prod.shop.com` avec un sous-ensemble de tests automatiques et quelques utilisateurs internes.
3. Décision « go » → modification du record DNS / ingress : `api.shop.com → green.prod.shop.com`. La bascule prend quelques secondes (propagation DNS) ou un instant (ingress K8s).
4. Si problème détecté dans les 30 minutes → rebascule sur Blue, on n'a perdu personne.
5. Si tout va bien, après 24h on supprime Blue (libération des ressources).

**Pré-requis** : la BDD doit être compatible avec les deux versions (migrations *backward-compatible*).

### 1.2.4 Canary

**Définition**

**Canary release** (« canari ») : on dirige un petit pourcentage du trafic (1, 5, 10 %) vers la nouvelle version. Si les métriques restent saines, on augmente progressivement (25, 50, 100 %). Si elles se dégradent, rollback immédiat. Le nom vient des canaris qu'on descendait dans les mines : si l'oiseau mourait, on savait qu'il fallait remonter.

**Exemple**

**Cas** : déploiement d'une nouvelle version du moteur de recommandation. On l'expose à 5 % des utilisateurs pendant 1 heure.
**Configuration Argo Rollouts (K8s) :**

```
strategy:
  canary:
    steps:
      - setWeight: 5 # 5% du trafic
      - pause: { duration: 1h }
      - setWeight: 25 # puis 25%
      - pause: { duration: 30m }
      - setWeight: 50
      - pause: { duration: 30m }
      - setWeight: 100
    analysis:
      templates:
        - templateName: error-rate
      args:
        - name: service-name
          value: recommend-api
```

**Critère automatique de rollback** : si le taux d'erreur de la nouvelle version dépasse 1 %, Argo annule et redirige 100 % vers l'ancienne. Aucune intervention humaine nécessaire la nuit.

### 1.2.5 A/B Testing

**Définition**

**A/B Testing** : ressemble au canary, mais le découpage du trafic vise à *tester une hypothèse produit* (« la variante B convertit-elle mieux ? »), pas à valider une version technique. Le routage se fait par *feature flag* : un même build embarque les deux variantes et un drapeau choisit laquelle servir par utilisateur.

**Exemple**

**Cas** : tester une nouvelle page de paiement.

- Variante A (existante) : 50 % des utilisateurs.
- Variante B (nouveau bouton « Acheter en un clic ») : 50 %.

**Outil :** LaunchDarkly, Unleash, Flagsmith, GrowthBook.

```
// dans le code
if (ldClient.variation("checkout-redesign", user, false)) {
  return <CheckoutB />;
} else {
  return <CheckoutA />;
}
```

**Métrique mesurée** : taux de conversion. Après deux semaines et 100 000 sessions, B convertit 12,3 % vs 11,1 % pour A → on adopte B.

### 1.2.6 Shadow (mirror traffic)

**Définition**

**Shadow** (ou *traffic mirroring*) : la nouvelle version reçoit une copie du trafic réel, mais sa réponse est *ignorée* (jetée) — seule la réponse de l'ancienne version est retournée à l'utilisateur. Permet de valider en charge réelle, avec les vrais payloads, sans aucun risque pour les utilisateurs.

**Exemple**

**Cas** : on veut tester en condition réelle la nouvelle implémentation Rust d'un endpoint critique avant de basculer.
**Configuration avec Envoy / Istio :**

```
http:
  - route:
      - destination: { host: orders-api, subset: v1 } # réponse servie
    mirror: { host: orders-api, subset: v2-rust } # copie ignorée
    mirrorPercentage: { value: 100.0 }
```

**À surveiller** : comparer côté observabilité les latences et erreurs des deux versions. Bonus : on détecte les régressions sans aucun impact utilisateur.

**Attention** : la copie déclenche les vrais effets de bord si on n'y prend pas garde (deux appels au PSP, deux e-mails envoyés). Toujours configurer un mode « lecture seule » côté shadow ou intercepter les sorties.

# 2 Observabilité : les 3 piliers

**Définition**

**Observabilité** : capacité à comprendre l'état interne d'un système à partir des données qu'il émet. Repose sur trois piliers complémentaires : *logs*, *métriques*, *traces*.

| Pilier | Question | Idéal pour |
|---|---|---|
| Logs | Que s'est-il passé ? | Investigation a posteriori |
| Métriques | Combien ? À quelle vitesse ? | Dashboards, alertes |
| Traces | Où le temps est-il passé ? | Debug d'architectures distribuées |

# 3 Logs structurés

## 3.1 Pourquoi structurer ?

**Définition**

**Log structuré** : ligne de log au format machine (JSON), avec des champs typés. Permet de filtrer, requêter et corréler les logs sans regex complexes.

**Exemple**

Une ligne de log structurée typique :

```
{
  "ts": "2026-05-05T10:14:02.341Z",
  "level": "INFO",
  "service": "orders-api",
  "version": "1.4.2",
  "env": "prod",
  "traceId": "4bf92f3577b34da6",
  "userId": 42,
  "method": "POST",
  "path": "/orders",
  "status": 201,
  "durationMs": 132
}
```

## 3.2 Bonnes pratiques

### 3.2.1 Format NDJSON

**Définition**

**NDJSON** (*Newline-Delimited JSON*) : convention où chaque ligne du fichier ou du flux est un objet JSON valide et autonome, séparé du suivant par un saut de ligne (`\n`). Pas de virgules entre objets, pas de tableau englobant. C'est le format *de facto* des logs structurés et des exports en streaming. Souvent rencontré aussi sous le nom `JSONL` (*JSON Lines*).

**Pourquoi ce format est idéal pour les logs ?**

- *Append-only* : on peut continuer à écrire des lignes sans réécrire le fichier.
- *Streaming-friendly* : on peut traiter ligne par ligne sans charger tout en mémoire (utile pour des fichiers de plusieurs gigaoctets).
- *grep-compatible* : `grep "ERROR" app.log` fonctionne immédiatement.
- *Tolérant à la corruption* : si une ligne est tronquée, les suivantes restent valides.

**Exemple**

**Un vrai fichier NDJSON** (`app.log`) :

```
{"ts":"2026-05-05T10:14:02.341Z","level":"INFO","msg":"login","userId":42}
{"ts":"2026-05-05T10:14:03.012Z","level":"WARN","msg":"slow query","ms":820}
{"ts":"2026-05-05T10:14:03.890Z","level":"ERROR","msg":"db timeout","traceId":"a1b2"}
```

**À ne PAS confondre avec un tableau JSON** (qui serait inutilisable pour des logs) :

```
[
  {"ts":"...","level":"INFO"},
  {"ts":"...","level":"WARN"}
]
```

Un tableau JSON exigerait de charger le tout et de retoucher les virgules à chaque ajout.

**Lecture en streaming Node :**

```
import { createReadStream } from "fs";
import { createInterface } from "readline";
const rl = createInterface({ input: createReadStream("app.log") });
for await (const line of rl) {
  const e = JSON.parse(line);
  if (e.level === "ERROR") console.log(e);
}
```

### 3.2.2 Champs standards (recommandation OpenTelemetry)

**Définition**

**OpenTelemetry** (souvent abrégé *OTel*) : projet open source de la *CNCF* (*Cloud Native Computing Foundation*) qui standardise la collecte de logs, métriques et traces. Définit un vocabulaire commun pour que les outils d'observabilité s'interconnectent (cf. section dédiée plus loin).

Les champs canoniques à inclure dans chaque ligne :

- `timestamp` : horodatage ISO 8601 avec milliseconde et fuseau UTC.
- `severity` : niveau de sévérité (DEBUG, INFO, WARN, ERROR, FATAL).
- `trace_id`, `span_id` : identifiants W3C Trace Context (propagation distribuée, cf. plus loin).
- `service.name` : nom du service émetteur (`orders-api`).
- `service.version` : version (`1.4.2`).

### 3.2.3 Niveaux de log cohérents

- **DEBUG** : détails internes utiles uniquement au développeur. Désactivé en production.
- **INFO** : événement métier normal (« utilisateur connecté », « commande créée »).
- **WARN** : situation anormale mais non bloquante (« 3 tentatives de retry »).

- **ERROR** : opération qui a échoué pour un utilisateur. Génère une alerte si fréquent.
- **FATAL** (ou CRITICAL) : le processus ne peut pas continuer. Alerte immédiate.

### 3.2.4 Jamais de PII dans les logs

**Définition**

**PII** (*Personally Identifiable Information*, en français « données à caractère personnel ») : toute information permettant d'identifier directement ou indirectement une personne physique. La notion est codifiée par le **RGPD** (Règlement Général sur la Protection des Données, UE, 2018) qui sanctionne fortement leur mauvais traitement.

**Exemples de PII :** nom, prénom, e-mail, numéro de téléphone, adresse postale, adresse IP, identifiant client, numéro de sécurité sociale, données biométriques, géolocalisation, photo de visage, numéro de carte bancaire, plaque d'immatriculation, identifiant publicitaire mobile.

**Pourquoi PII et logs ne font pas bon ménage :**

- Les logs sont souvent répliqués, exportés vers des SaaS externes (Datadog, Splunk), conservés des années.
- Une faille sur l'agrégateur de logs = fuite massive de PII.
- Le droit à l'oubli RGPD (art. 17) est très difficile à appliquer sur des logs.

**Exemple**

**MAL :**

```
{"msg":"login","email":"alice@exemple.fr","ip":"82.231.45.12","cb":"4970-...-0042"}
```

**BIEN :**

```
{"msg":"login","userId":42,"ipHash":"a9c1b2","cbLast4":"0042"}
```

**Techniques :** hachage (`sha256(ip + salt)`), troncature (4 derniers chiffres), pseudonymisation (`userId` interne au lieu de l'e-mail), masquage côté logger (`pino-noir`, `winston` avec format custom, `python-json-logger` avec filtres).

### 3.2.5 Propagation du trace ID

**Définition**

**Trace ID** : identifiant unique (16 octets, soit 32 caractères hex) qui suit une requête à travers tous les services qu'elle traverse. Standard W3C *Trace Context* via l'en-tête HTTP `traceparent`. Permet, dans Kibana ou Grafana, de filtrer tous les logs liés à une même requête utilisateur, même si elle a traversé 12 microservices.

**Exemple**

**Header HTTP standard :**

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
             ^^ ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ ^^^^^^^^^^^^^^^^ ^^
          ver trace-id (16 octets) span-id (8 oct) flags
```

**Effet en Kibana :** requête `traceId:"4bf92f3577b34da6a3ce929d0e0e4736"` → reconstitution complète : `api-gateway → auth-service → orders-api →`

`stock-service` → `mail-service`, avec les latences et erreurs de chacun.

# 4 Métriques avec Prometheus

## 4.1 Architecture

**Définition**

**Prometheus** (CNCF, créé par SoundCloud en 2012) : système de monitoring open source basé sur un *modèle pull* : Prometheus interroge périodiquement les services pour collecter leurs métriques exposées sur l'endpoint `/metrics`.

- **Targets** : services exposant `/metrics`.
- **Scrape** : Prometheus va lire les targets périodiquement (15s par défaut).
- **TSDB** (*Time Series Database*) : stockage local optimisé pour les séries temporelles.
- **Alertmanager** : composant qui reçoit les alertes, les déduplique, les route.

## 4.2 Types de métriques Prometheus

- **Counter** : valeur monotone croissante (nombre de requêtes total).
- **Gauge** : valeur instantanée (mémoire utilisée, file d'attente).
- **Histogram** : répartition en buckets, permet de calculer des quantiles.
- **Summary** : quantiles calculés côté client. Moins flexibles que les histograms.

## 4.3 Méthode RED

**À retenir**

Pour instrumenter une API REST, capturer au minimum :

- **R — Rate** : requêtes par seconde, par endpoint et par code de statut.
- **E — Errors** : taux d'erreur (4xx et 5xx).
- **D — Duration** : latence sous forme d'histogramme, permettant les quantiles p50/p95/p99.

## 4.4 PromQL — requêtes essentielles

**Exemple**

```
# Requêtes par seconde sur 5 minutes
rate(http_requests_total[5m])

# Taux d'erreur 5xx en pourcentage
100 * sum(rate(http_requests_total{status=~"5.."}[5m]))
    / sum(rate(http_requests_total[5m]))

# Latence p95 par route
histogram_quantile(0.95,
  sum by (le, route) (rate(http_request_duration_seconds_bucket[5m])))
```

```
# Top 5 endpoints les plus lents (p99)
topk(5, histogram_quantile(0.99,
  sum by (le, route) (rate(http_request_duration_seconds_bucket[5m]))))
```

# 5 Grafana et dashboards

**Définition**

**Grafana** : outil open source de visualisation et d'alerting. Multi-datasource (Prometheus, Loki, Tempo, Elasticsearch, MySQL, etc.). Standard de fait pour les dashboards d'observabilité.

## 5.1 Composer un dashboard utile

Penser *audience* :

- **Direction** : SLO, disponibilité, error budget.
- **Ops** : santé infra (RED + USE), alertes en cours.
- **Dev** : par endpoint, traces, logs.

**Bonnes pratiques.**

- Une vue par audience, pas un dashboard de 50 panneaux.
- Variables Grafana (`$env`, `$service`) pour réutiliser un dashboard.
- Annotations pour marquer les déploiements et les incidents.
- Versionner les dashboards en JSON dans Git.

# 6 SLO et error budget

**Définition**

**SLI** (*Service Level Indicator*) : la mesure (par exemple : % de requêtes < 200 ms réussies).

**SLO** (*Service Level Objective*) : la cible interne (par exemple : 99,9 % sur 30 jours).

**SLA** (*Service Level Agreement*) : contrat client, plus permissif que le SLO interne.

**Error budget** : 1 - SLO. Pour 99,9 % : 0,1 % = 43,2 minutes de budget d'erreur sur 30 jours.

## 6.1 Alerting basé sur SLO

**Règle d'or.** Alerter sur les *symptômes utilisateur*, pas sur les *causes* (« CPU > 80 % » est une cause possible mais pas un symptôme : l'utilisateur s'en moque tant que la latence reste basse).

**Multi-window multi-burn-rate** (Google SRE) :

- Fenêtre courte (5 min) + burn rate élevé → alerte page.
- Fenêtre longue (1 h) + burn rate élevé → confirme.
- Fenêtre encore plus longue (6 h) + burn rate modéré → ticket.

# 7 Tracing distribué

**Définition**

**Tracing distribué** : technique qui permet de suivre une requête à travers plusieurs services et opérations (BDD, MQ, cache), pour identifier où le temps est passé.

**Trace** : représente une requête de bout en bout.
**Span** : une opération dans la trace (un appel HTTP, une requête BDD).

## 7.1 OpenTelemetry

**Définition**

**OpenTelemetry (OTel)** : standard CNCF d'observabilité, fusion d'OpenTracing et OpenCensus. Trois composants : API stable, SDK par langage, Collector qui reçoit/transforme/exporte vers un backend.

**Standard *traceparent*** (W3C Trace Context) : en-tête HTTP qui propage l'identifiant de trace entre services.

# 8 ELK Stack et alternatives

## 8.1 Composants ELK

- **Beats / Filebeat** : agents légers qui collectent les logs.
- **Logstash** : pipeline de transformation (parsing, enrichissement).
- **Elasticsearch** : moteur de recherche full-text.
- **Kibana** : UI pour rechercher, créer des dashboards, alerter.

## 8.2 Alternatives

- **OpenSearch** : fork open source d'Elasticsearch maintenu par AWS depuis 2021.
- **Grafana Loki** : indexe seulement les labels (pas le contenu). Beaucoup moins cher en stockage.

# 9 Les outils du module

## 9.1 Prometheus

**Outil**

**Prometheus** (`https://prometheus.io`) : système de monitoring de séries temporelles, collecte par scrape périodique. Intégré nativement à Kubernetes et au CNCF. Apprend en quelques heures, opère pendant des années.

## 9.2 Grafana

**Outil**

**Grafana** (`https://grafana.com`) : visualisation et alerting multi-datasource. Open source en version OSS, version Cloud en SaaS. Standard de fait pour les dashboards.

## 9.3 Elastic Stack / OpenSearch

**Outil**

**Elastic Stack** (`https://www.elastic.co`) ou **OpenSearch** (`https://opensearch.org`) : plateformes de recherche full-text + analyse de logs. Très puissants mais coûteux en stockage.

## 9.4 Grafana Loki

**Outil**

**Loki** (Grafana Labs) : alternative open source à Elasticsearch pour les logs, pensée pour Kubernetes. Bien moins gourmand en disque que Elastic. Langage de requête *LogQL* (proche de PromQL).

## 9.5 OpenTelemetry

**Outil**

**OpenTelemetry** (`https://opentelemetry.io`) : standard CNCF d'instrumentation. Auto-instrumentation pour Java, Python, Node, Go, .NET, etc. Indispensable pour ne pas se lier à un fournisseur APM.

## 9.6 Jaeger, Tempo, Zipkin

**Outil**

**Jaeger** (Uber, CNCF), **Tempo** (Grafana Labs), **Zipkin** (Twitter) : backends de tracing distribué. Reçoivent les traces, les stockent, les exposent dans une UI. Tempo est le plus économique en stockage.

## 9.7 ArgoCD, Flux

**Outil**

**ArgoCD** et **Flux** : outils de *GitOps* (déploiement déclaratif depuis Git). Le cluster Kubernetes se synchronise automatiquement avec un dépôt Git de référence.

### 9.8 Trivy, Grype

**Outil**

**Trivy** (Aqua Security) et **Grype** (Anchore) : scanners de vulnérabilités sur les images container et les dépendances. À intégrer en CI pour bloquer les builds avec des CVE critiques.

## 10 Synthèse

**À retenir**

- Trois piliers : logs (qui), métriques (combien), traces (où dans la chaîne).
- Méthode **RED** côté API (Rate, Errors, Duration). Méthode **USE** côté infra.
- Logs structurés JSON, traceId propagé, jamais de PII ni secrets.
- Prometheus pull + PromQL. Buckets d'histogrammes adaptés au domaine.
- Alerter sur les SLO et les symptômes utilisateur, pas sur le CPU brut.
- OpenTelemetry pour ne pas se lier à un fournisseur APM.

## 11 Lexique du module

**APM** Application Performance Monitoring.

**Cardinalité** Nombre de séries distinctes générées par les labels d'une métrique.

**Counter** Métrique Prometheus monotone croissante.

**CI/CD** Continuous Integration / Continuous Delivery / Continuous Deployment.

**DAST** Dynamic Application Security Testing.

**ELK** Elasticsearch + Logstash + Kibana.

**Error budget** Marge d'erreur tolérée par un SLO.

**Gauge** Métrique Prometheus instantanée.

**Histogram** Métrique Prometheus à buckets, permet le calcul de quantiles.

**Loki** Système de logs Grafana, indexation par labels uniquement.

**NDJSON** Newline-Delimited JSON.

**Observabilité** Capacité à comprendre l'état interne d'un système.

**OpenTelemetry** Standard CNCF d'instrumentation.

**PromQL** Langage de requête Prometheus.

**Quantile** Valeur en-deçà de laquelle se trouve un pourcentage donné des observations.

**RED** Rate, Errors, Duration (méthode pour les services).

**SAST** Static Application Security Testing.

**SBOM** Software Bill of Materials.

**Scrape** Prometheus va lire les métriques des targets périodiquement.

**SLA** Service Level Agreement (engagement contractuel).

**SLI** Service Level Indicator (la mesure).

**SLO** Service Level Objective (la cible interne).

**Span** Une opération dans une trace.

**TSDB** Time Series Database.

**Trace** Suivi d'une requête de bout en bout.

**USE** Utilization, Saturation, Errors (méthode pour les ressources).

## 12 Pour aller plus loin

- Google *SRE Book* et *SRE Workbook* (gratuits en ligne).
- Cindy Sridharan, *Distributed Systems Observability* (gratuit, 50 pages).
- Brendan Gregg, *Systems Performance* (méthode USE).
- Alex Hidalgo, *Implementing Service Level Objectives*.
- Documentation officielles : prometheus.io, grafana.com, opentelemetry.io.